Hitesh Rathore (HR) Notes
LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

CSS CASCADING

HOW CSS RULES COMBINE AND OVERRIDE EACH OTHER

Cascading :-

```
<body>
    <h1 class="first-class" id="first-id">Hello Dears</h1>
    <ul>
        <li>Lallu</li>
        <li>Kallu</li>
    </ul>
</body>
```

~style>

```
h1 {
        background-color: pink;
}
.first-class {
        background-color: blue;
}
#first-id {
        background-color: green;
}
```

OR

```
h1 {
        background-color: pink;
}
#first-id {
        background-color: green;
}
.first-class {
        background-color: blue;
}
```

If background exists, which one will happen??
= Cascading will tell us deciding factors;
→ Priority Order: order < specificity < Type < Important

1) ORDER IN CSS (WHEN SELECTING BY THE SAME SELECTOR)

EXAMPLE 1

```
h1 {
    background-color: pink;
    background-color: blue;
}
```

Final Applied Color: **BLUE**

OR

EXAMPLE 2

```
h1 {
    background-color: pink;
}
h1 {
    background-color: blue;
}
```

Final Applied Color: **BLUE**

EXPLANATION
If we are selecting by the same selector, then order matters and the last one wins.

SUMMARY
When the same selector is used multiple times,
the last declared rule is applied.

2) SPECIFICITY IN CSS

PRIORITY ORDER (Higher → Lower)

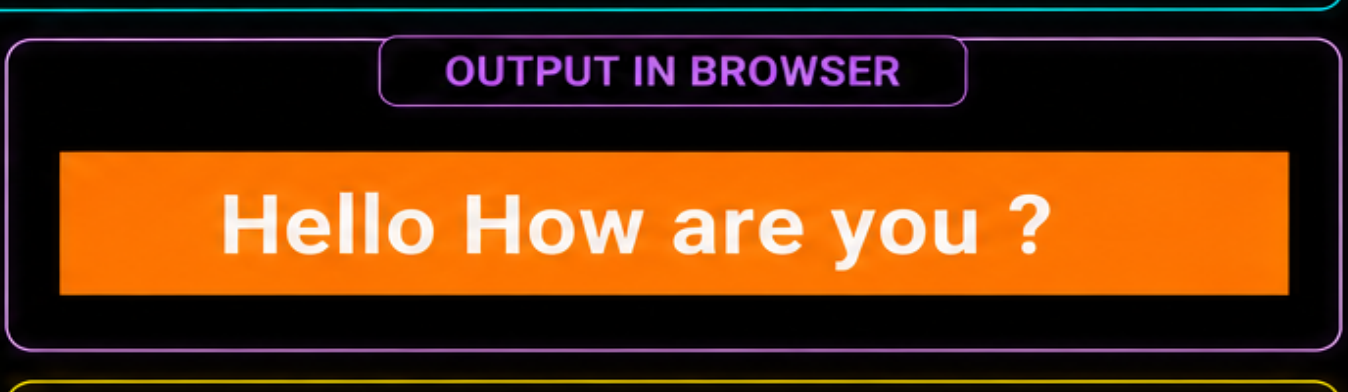
① ID # > ② ATTRIBUTE [ ] > ③ CLASS . > ④ CHILD & DESCENDANT > > ⑤ ELEMENT p

- The more specific a selector is, the higher its priority
- Child and descendant selectors have the same priority, so we will look at the order.

EXAMPLE (HTML + CSS)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <style>
      .box{
            background-color: pink;
      }
      #first{
            background-color: orange;
      }
      h1{
            background-color: blue;
      }
  </style>
</head>
<body>>
   <h1 class="box" id="first">Hello How are you ?</h1>
</body>
</html>
```

OUTPUT IN BROWSER



Hello How are you ?

BACKGROUND COLOR

ORANGE

Here, the #first selector (ID selector) will be applied because it is more specific than .box (class selector) and h1 (element selector).
Hence, the background color is ORANGE.

3) TYPE IN CSS

PRIORITY: Inline > Internal , External →Order will matter in case of Internal && External

INLINE CSS
Written inside an element using the style attribute.

INTERNAL CSS
Written inside the <style> tag within the <head> section.

EXTERNAL CSS
Written in a separate .css file and linked using <link>.

EXAMPLE 1 Internal CSS is written after External CSS (in HTML)

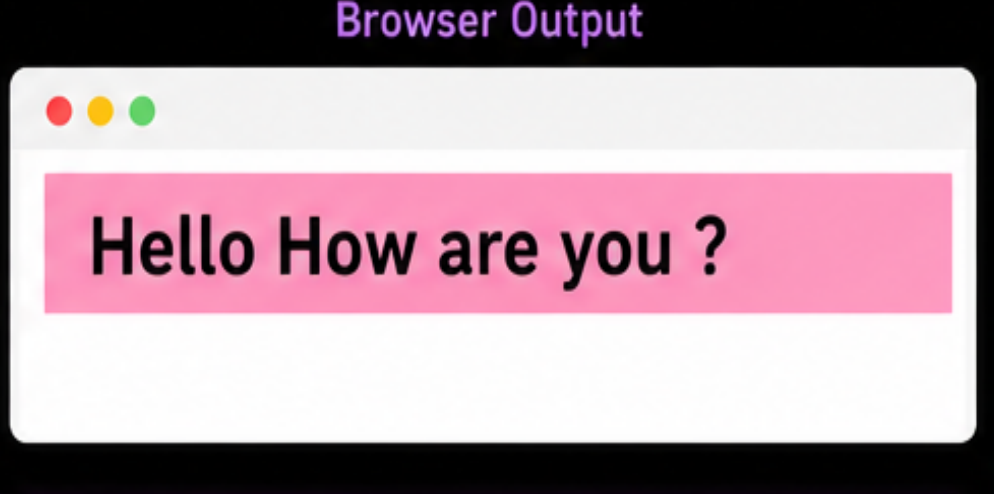
index.html

```
<!DOCTYPE html>
<html lang="en">
 <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="./style.css">
    <style>
       h1{
          background-color: pink;
       }
  </style>
<body>
    <h1>Hello How are you ?</h1>
</html>
```

style.css (External CSS)

```
h1{
   background-color: blue;
}
```

Browser Output



Hello How are you ?

Since Internal CSS is written later in the HTML, it will be applied.
Hence, background color is PINK.

EXAMPLE 2 External CSS (link) is written after Internal CSS (in HTML)

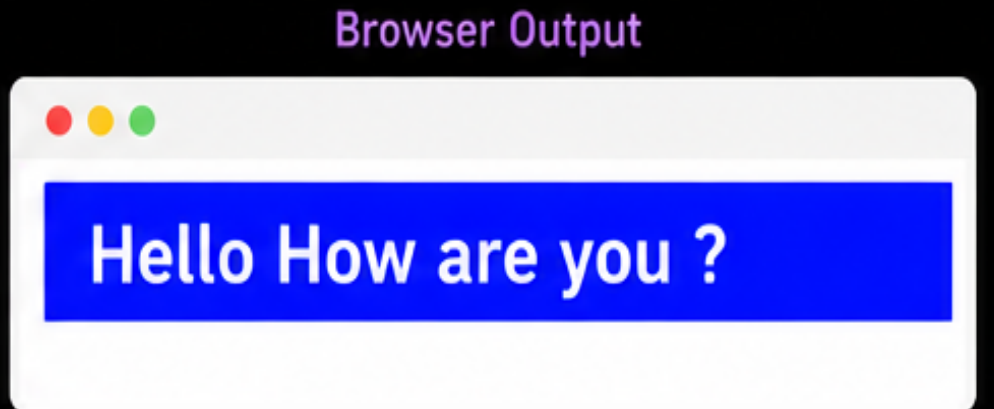
index.html

```
<!DOCTYPE html>
<html lang="en">
 <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        h1{
           background-color: pink;
        }
    </style>
   <link rel="stylesheet" href="./style.css">
</head>
<body>
   <h1>Hello How are you ?</h1>
</html>
```

style.css (External CSS)

```
h1{
   background-color: blue;
}
```

Browser Output



Hello How are you ?

Since External CSS (link) is written later in the HTML, it will be applied.
Hence, background color is BLUE.

4) !IMPORTANT IN CSS

!important is used to give high priority to a CSS property.

HOW IT WORKS

- The property with !important will be applied, no matter its type (Inline, Internal, External).
- If more than one declaration of the same property has !important, then the one with higher specificity will win.
- If specificity is also same, then order (written later) will win.

EXAMPLE 1 External CSS has !important

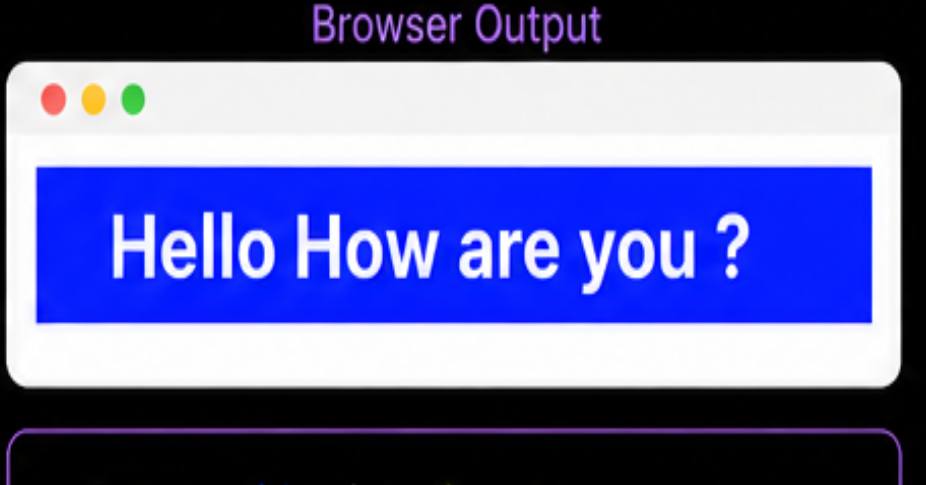
index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <link rel="stylesheet" href="style.css">
   <style>
       h1 { background-color: pink; }  /* Internal CSS */
   </style>
</head>
<body>
   <h1>Hello How are you ?</h1>
</body>
</html>
```

style.css (External CSS)

```
h1 {
   background-color: blue !important;
}
```

Browser Output



Hello How are you ?

Because blue has !important, it will be applied over pink.

EXAMPLE 2 Internal CSS has !important (written later)

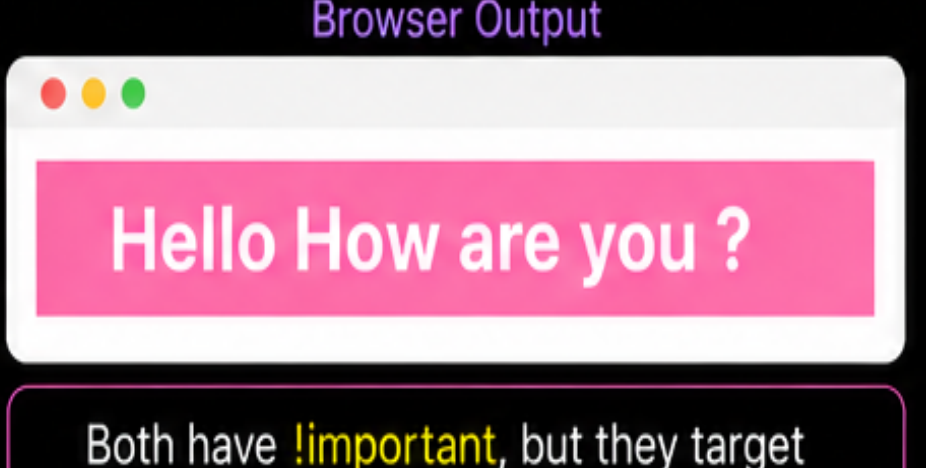
index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <link rel="stylesheet" href="style.css">
   <style>
     h1 { background-color: pink !important; }  /* Internal CSS */
   </style>
</head>
<body>
   <h1>Hello How are you ?</h1>
</body>
</html>
```

style.css (External CSS)

```
h1 {
   background-color: blue !important;
}
```

Browser Output



Hello How are you ?

Both have !important, but they target the same selector (h1) with same specificity, so order matters.
Internal CSS is written later, so pink wins.

Summary: !important gives the highest priority. If multiple !important rules exist, then specificity decides, and if specificity is same, the one written later wins.

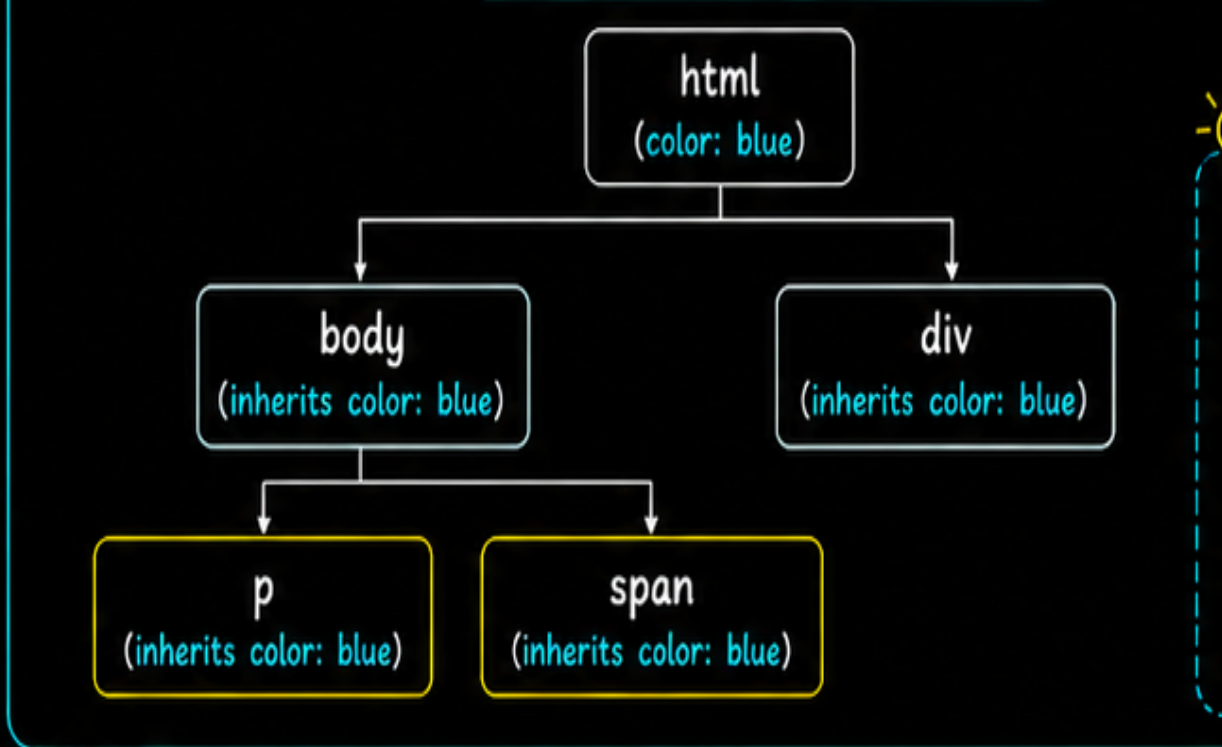
INHERITANCE IN CSS

Inheritance in CSS means that some properties applied to a parent element are automatically taken by its child elements.

HOW IT WORKS

- ✓ When a CSS property is set on a parent element, the child elements take it automatically (if that property is inheritable).
- ✓ Not all properties are inherited.
- ✓ Inheritance flows from parent → child → grandchild and so on.

VISUAL REPRESENTATION



If a child has its own value for that property, then it will not inherit from the parent.

PROPERTIES

INHERITED PROPERTIES (Automatically passed to children)	NON-INHERITED PROPERTIES (Not passed to children)
color	background
font-family	margin
font-size	padding
font-weight	border
font-style	width
line-height	height
text-align	display
visibility	position
cursor	top / left / right / bottom
letter-spacing	overflow
word-spacing	etc.
etc.	

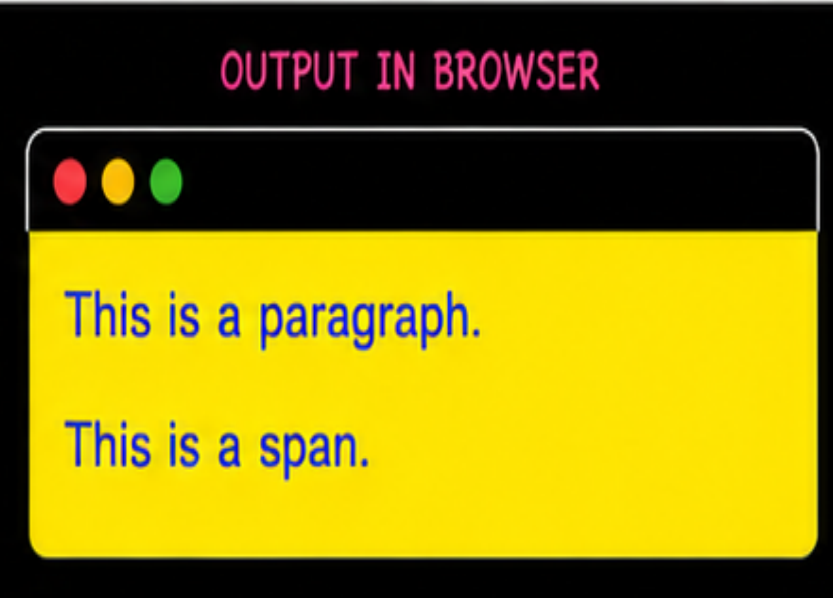
You can make a non-inherited property inherit using: inherit;

EXAMPLE

HTML

```
<!DOCTYPE html>
<html>
  <head>
   <style>
      body {
         color: blue;            /* inherited */
         font-size: 20px;        /* inherited */
         background: yellow;     /* not inherited */
      }
   </style>
</head>
<body>
   <div>
      <p>This is a paragraph.</p>
      <span>This is a span.</span>
   </div>
</body>
</html>
```

OUTPUT IN BROWSER



This is a paragraph.

This is a span.

- color and font-size are inherited (hence text is blue & 20px).
- background is NOT inherited (hence no yellow background in child).

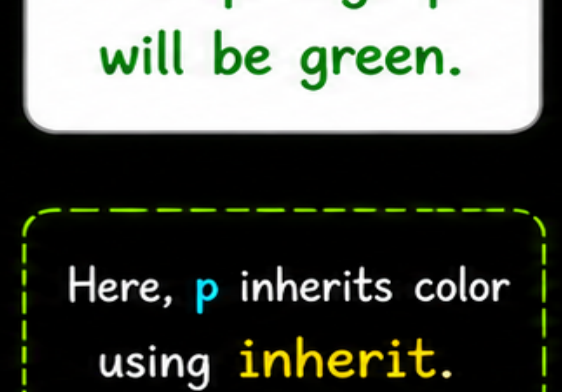
USING inherit KEYWORD

We can force a property to be inherited using the inherit keyword.

```
<style>
  .box {
     color:  green;
  }
  .box p {
     color: inherit;    /* takes green from .box */
  }
</style>

<div class="box">
   <p>This paragraph will be green.</p>
</div>
```

OUTPUT



This paragraph will be green.

Here, p inherits color using inherit.

SUMMARY

- Inheritance makes CSS more powerful and reduces repetition.
- Only specific properties are inherited.
- If both parent and child have the same property, child's value will override the inherited value.
- Use inherit keyword to explicitly inherit a property.